

TECHNICAL PROJECT REPORT

# Sanskrit Retrieval-Augmented Generation (RAG) System

---

Prepared by

**Harshvardhan Lambe**

*AI/ML Engineering & RAG Architecture*

## 01 EXECUTIVE SUMMARY

This project implements a fully offline, CPU-optimized Sanskrit Retrieval-Augmented Generation (RAG) system designed specifically for Sanskrit document understanding and intelligent semantic retrieval. The architecture was engineered using modern AI engineering practices emphasizing modularity, scalability, low resource consumption, retrieval accuracy, and hallucination reduction.

**The system integrates the following core capabilities:**

- Multilingual semantic embeddings for cross-language understanding
- Vector-based retrieval with ChromaDB for fast local search
- CPU-efficient LLM inference using quantized lightweight models
- Streamlit-based interactive querying interface
- Unicode-aware Sanskrit preprocessing pipeline

## 02 SYSTEM ARCHITECTURE & WORKFLOW

The system follows a modular Retrieval-Augmented Generation architecture consisting of eight integrated layers, each responsible for a discrete stage of the pipeline:

- **Document Ingestion Layer** — Documents (.txt/.pdf) are parsed using PyPDFLoader or TextLoader
- **Sanskrit Preprocessing Layer** — Handles Unicode normalisation, OCR cleanup, and symbol handling
- **Chunking & Embedding Layer** — Semantic chunks are embedded via paraphrase-multilingual-MiniLM-L12-v2 and stored in ChromaDB
- **ChromaDB Vector Storage** — Lightweight, local persistent vector database
- **Semantic Retrieval Engine** — Queries trigger cosine similarity search retrieving the Top-5 relevant context chunks
- **Prompt Construction Pipeline** — Retrieved chunks are injected into structured prompt templates
- **CPU-Optimized LLM Generator** — A local Qwen2.5-0.5B-Instruct LLM generates responses strictly based on retrieved context
- **Interactive Streamlit Frontend** — User-facing interface for querying and result display

**System Workflow:** Documents → Preprocessing → Chunking → Embeddings → ChromaDB → Query → Retrieval → Prompt Injection → LLM → Response

### 03 TECHNOLOGY STACK

All components were selected for CPU efficiency, open-source availability, and production-grade reliability:

| Component       | Technology Used          | Purpose                               |
|-----------------|--------------------------|---------------------------------------|
| Framework       | LangChain                | RAG pipeline orchestration            |
| Vector Database | ChromaDB                 | Lightweight local vector search       |
| Embedding Model | MiniLM Multilingual      | CPU-efficient multilingual embeddings |
| LLM             | TinyLlama / Qwen2.5-0.5B | Quantized lightweight inference       |
| Frontend        | Streamlit                | Interactive local UI                  |
| PDF Parsing     | PyMuPDF                  | Fast PDF text extraction              |

### 04 SANSKRIT PREPROCESSING PIPELINE

Sanskrit introduces unique NLP challenges due to compound words (Sandhi), Unicode complexity, and OCR inconsistencies in scanned manuscripts. The preprocessing pipeline addresses these systematically:

- UTF-8 Unicode normalisation
- Whitespace and symbol cleaning
- OCR artifact removal from scanned PDFs
- Recursive semantic chunking with context overlap
- Transliterated query support (Devanagari ↔ Latin)
- Sanskrit Danda (|) separator support to maintain shloka integrity

#### Chunking Strategy

**Chunk Size:** 250–400 characters

**Chunk Overlap:** 50–80 characters

This strategy improved semantic retrieval precision while preserving contextual continuity across compound Sanskrit terms and multi-line shlokas.

## 05 RETRIEVAL & GENERATION MECHANISM

The retrieval pipeline uses multilingual semantic embeddings combined with cosine similarity search to bridge the gap between English queries and Sanskrit text corpora.

### Retrieval Flow

- User query converted into dense vector embeddings
- Cosine similarity search performed across the ChromaDB vector index
- Top-K relevant chunks retrieved with configurable K parameter
- Retrieved context injected into structured prompt template
- LLM generates a grounded, contextual response strictly from retrieved data

### Hallucination Reduction Techniques

- Strict context-only prompting — no external knowledge injection
- Top-K retrieval optimisation with tuned similarity thresholds
- Quantized inference stability via fp32 precision
- Context injection constraints to prevent prompt dilution

### Cross-Language Bridge

The multilingual embedding model (paraphrase-multilingual-MiniLM-L12-v2) allows English queries to effectively retrieve Sanskrit text. The Qwen2.5-0.5B model is optimized for CPU-only inference using low\_cpu\_mem\_usage and fp32 precision settings.

## 06 PERFORMANCE EVALUATION

Benchmarks conducted on a standard consumer-grade CPU machine with no GPU acceleration:

| Metric               | Performance               |
|----------------------|---------------------------|
| Document Loading     | 1–3 seconds               |
| Embedding Generation | 3–5 seconds               |
| Retrieval Latency    | < 50 milliseconds         |
| LLM Generation       | 2–5 seconds               |
| RAM Usage            | < 3 GB                    |
| GPU Requirement      | Not Required ✓            |
| Retrieval Quality    | High Semantic Relevance ✓ |

## 07 CHALLENGES FACED

- Limited availability of Sanskrit-specific pre-trained embedding models
- Complex Sanskrit compound word structures (Sandhi Viched) impeding clean tokenisation
- CPU-only inference limitations affecting throughput for large corpora
- OCR noise and font inconsistencies in scanned PDF manuscripts
- Balancing retrieval quality with constrained memory footprint

## 08 FUTURE IMPROVEMENTS

- Hybrid Retrieval combining BM25 sparse search with dense vector search
- Advanced neural reranking pipelines (e.g., cross-encoder models)
- Sanskrit-specialized fine-tuned embedding models
- Conversation memory integration for multi-turn querying
- Multimodal OCR support for scanned manuscript images
- Distributed vector database scaling for large corpora

## 09 SANSKRIT DOCUMENT CORPUS

The following Sanskrit texts were used as the primary retrieval corpus for system evaluation:

- **Murkha-bhrtyasya** — A story about a foolish servant named Shankhanada; used to evaluate contextual character retrieval
- **Catusya Kalidasasya** — Tales of King Bhoja's court and Kalidasa's linguistic brilliance; rich in compound Sanskrit constructs
- **Vrddhayah Carturyam** — The 'Ghantakarna' mystery where an old woman discovers monkeys are responsible for a 'demon's' bell; narrative inference test
- **Bhaktasya Jnanodayah** — A moral tale about a devotee learning that God helps those who help themselves; semantic moral reasoning evaluation

## 10 CONCLUSION

This project demonstrates the implementation of a production-oriented Sanskrit Retrieval-Augmented Generation system capable of running entirely on CPU infrastructure. The architecture successfully integrates semantic retrieval, multilingual embeddings, vector databases, and lightweight LLM inference into a scalable and modular pipeline.

### Core competencies demonstrated through this project:

- End-to-end RAG architecture design and implementation
- Prompt engineering for hallucination control and context grounding
- CPU optimisation strategies for production deployment
- Vector search implementation with ChromaDB
- Unicode-aware NLP preprocessing for low-resource languages
- Local LLM deployment with quantization techniques

Prepared by:

**Harshvardhan Lambe**